

Практическая работа Тема 2. Особенности выполнения арифметических и логических операций в RISC процессорах

Логические сигналы в системах регулирования характеризуют состояния объектов, соответствие параметров допустимым диапазонам, отсутствие аварийных ситуаций. Они выполняют функции управляющих кодов, определяющих выполнение определенных действий. Каждый разряд управляющего кода может разрешать (или запрещать) выполнение определенных команд или процедур.

Логические операции в микропроцессорах предназначены для формирования и обработки логических сигналов, которые принимают значения 0 или 1. **Логические операции** выполняются поразрядно над 8-разрядными операндами с использованием регистровой или непосредственной адресации и формируют результаты в виде 8-разрядных слов. Результат операции в каждом разряде определяется в соответствии с таблицей 2.1. Для проверки правильности выполнения логических операций необходимо представить операнды и результат в двоичной системе счисления.

Таблица 2.1 – таблицы истинности логических операций

x	y	and x,y	or x,y	eor x,y
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

В командах с регистровой адресацией указаны два регистра, где расположены операнды, результат помещается в первый из них, а его предыдущее содержимое теряется. В одноадресных командах арифметических операций после мнемоники указан регистр, содержимое которого модифицируется командой.

Команды с непосредственной адресацией в качестве второго операнда содержат байт данных, а в мнемониках – букву *i* (от Immediate – непосредственно). Команды с непосредственной адресацией используют для ввода в программу исходных данных, они удобны при работе с константами. В ряде случаев в логических операциях выделяют один из операндов, считают его основным и называют «данные», а другой считают вспомогательным и называют «маска».

Признаки результата, формируемые при выполнении арифметических и логических операций, записываются в регистр состояния процессора – Status Register – SREG (рис. 2.1) и используются в командах условных переходов. Наиболее употребительны признаки C, Z, N.

Бит 0 – **C** (carry) Признак переноса. C=1, если при выполнении операции был перенос из старшего разряда результата. Перенос возникает также при сдвигах.

Бит 1 – **Z** (zero) Признак нуля. Z=1, если результат операции равен 0.

Бит 2 – N (negative). Признак отрицательного результата. Устанавливается, если старший (знаковый) бит результата равен 1.

Задание 2.1. Изучите примеры использования логических операций, приведенные в программе LR_2. Для ввода программы создайте новый файл в существующем проекте.

1. Откройте в AVR Studio файл предыдущей работы LR_1. Командой Save As запишите его с именем LR_2.

2. В окне Project укажите Source Files. Из меню по правой кнопке выберите Add Files to Project и укажите LR_2, «Открыть». Имя LR_2 появится в окне Project.

3. В окне Project укажите LR_2, по правой кнопке укажите Set As Entry Files.

4. Удалите старый файл ДК_1.

В отчете необходимо представить результаты выполнения программы в виде таблицы, формат которой приведен в работе 1.

Таблица 2.2 – пример для анализа логических операций

; Example 2.1	ldi r18, 0x67 ;12	and r18, r16 ;24
.include "m328pdef.inc";1	ldi r19, 0x89 ;13	or r17, r18 ;25
ldi r16, 0x08 ;2	ldi r20, 0xr18 ;14	bst r16, 7 ;26
out sph, r16 ;3	and r17, r16 ;15	bld r20, 2 ;27
ldi r16, 0xff ;4	andi r17, 0b00000010;16	sei ;28
out spl, r16 ;5	or r18, r16 ;17	cli ;29
m: call pp1 ;6	tst r16 ;18	ret ;30
; call pp2 ;7	eor r19, r16 ;19	pp2: ;31
; call pp3 ;8	eor r19, r19 ;20	ret ;32
jmp m ;9	mov r17, r20 ;21	pp3: ;33
pp1: ldi r16, 0x0f ;10	and r17, r16 ;22	ret ;34
ldi r17, 0x45 ;11	swap r16 ;23	

Строки 1 – 9 - из работы 1. В строках 10-14 выполняется инициализация – пересылка констант в регистры командами с непосредственной адресацией.

Операция «И» (and или andi) применяется в следующих случаях.

1) Сброс в 0 отдельных разрядов регистра с помощью маски. Разряды регистра, которым в маске (r16) соответствует 0, будут сброшены в 0. В строке 15 старшая тетрада регистра r17 сбрасывается.

2) Проверка значений отдельных бит в регистре путем анализа признака нуля «Z» после выполнения операции «И» с маской. Нулевой результат операции «И» означает, что в регистре равны нулю все разряды, для которых в маске записана «1». После выполнения строки 16 формируется z=1, это означает, что разряд 1, для которого в маске записана «1», в регистре r17 до операции был равен «0». Для наглядности маска представлена в двоичной системе счисления.

Операция «ИЛИ» (or, или ori) позволяет выполнить действия.

1) Установка в «1» отдельных бит в регистре с помощью маски. Биты регистра, которым в маске соответствует «1», будут установлены в «1». В строке 17 в младшей тетраде регистра r18 установятся единицы.

2) Тестирование – неразрушающая проверка перед операцией условного перехода, установка признаков Z, N выполняется командой «tst a», которая эквивалентна команде «ог a, a», но записывается короче. Значение числа при этом сохраняется без изменения. После выполнения строки 18 в SREG подучим признаки $Z = 0$, $N = 0$, так как в регистре r16 содержится число 0x0f.

Операция «сумма по модулю два» (eor) выполняет действия.

1) Инверсия определенных бит слова посредством маски. Биты регистра, которым в маске соответствует «1», инвертируются (строка 19).

2) Очистка регистра (строка 20).

3) Проверка двух слов на абсолютное равенство. В единственном случае, когда операнды поразрядно совпадают, результат операции «eor» равен нулю, о чем свидетельствует признак $Z = 1$.

Упаковка - составление одного слова из полей других слов. Вначале поля, которые необходимо использовать, выделяются посредством операции «И» с маской, затем объединяются операцией «ИЛИ». В строках 31-34 содержимое регистра r17 формируется из тетрад, принадлежавших регистрам r17 и r18. В строке 32 использована команда перестановки тетрад в байте.

Бит пользователя «t» содержится в регистре состояния процессора. В системе команд предусмотрена запись в бит «t» значения любого бита любого РОН (bst – Bit Store, строка 36), загрузки бита «t» в любой разряд РОН (bld – Bit Load, строка 37,), а также условные переходы по значению «t».

Для всех битов регистра состояния процессора предусмотрены команды установки и очистки, например, флагу разрешения прерываний I будут соответствовать команды SEI (Set Interrupt) – разрешить прерывания и CLI (Clear Interrupt) – запретить прерывания (строки 28, 29).

Неразрушающая проверка – операция сравнения двух чисел выполняется командами CP - Compare, CPC - Compare with Carry, CPI - Compare with Immediate. Выполняется вычитание чисел, формируются признаки, но результат вычитания не фиксируется и исходные числа не изменяются.

Команды сдвигов позволяют побитно сдвигать код операнда вправо (в сторону младших разрядов) или влево, бывают трех типов: логические, арифметические или циклические.

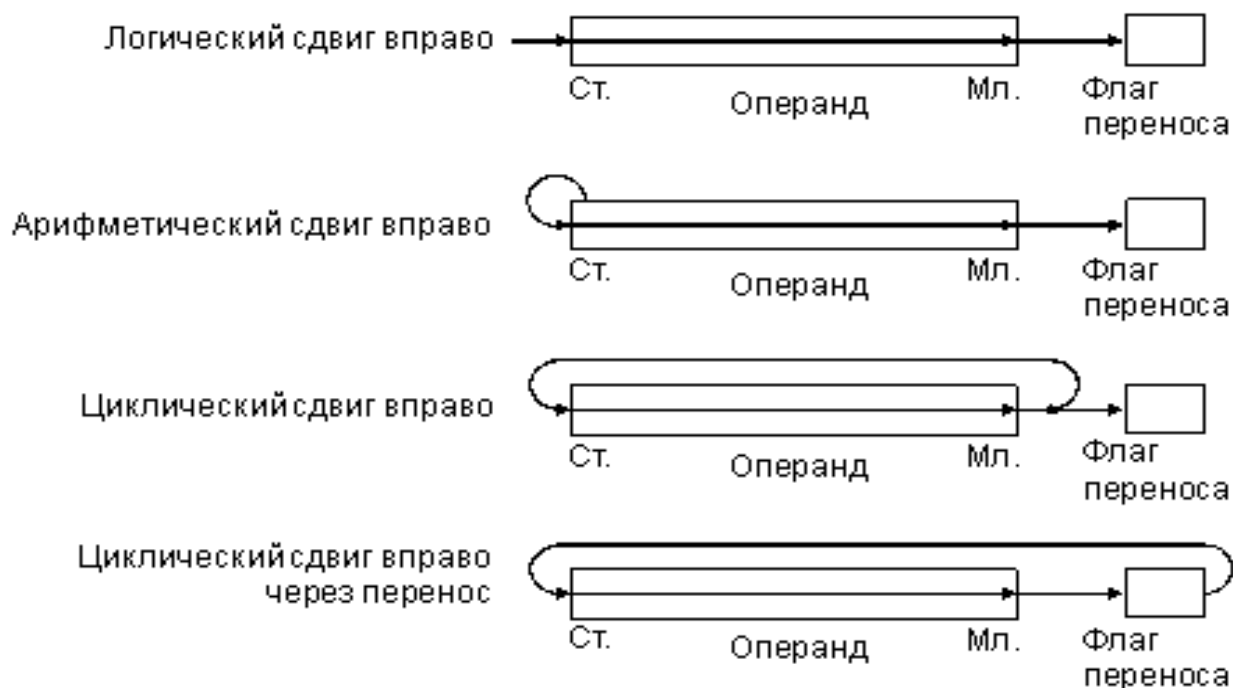


Рисунок 2.1 – Операции сдвигов

При логических сдвигах выдвигаемый бит записывается в качестве признака переноса в регистр состояния процессора, а освобождающийся бит заполняется нулем. Положительные числа при логическом сдвиге вправо делятся на 2, а при сдвиге влево – умножаются на 2. Арифметический сдвиг вправо делит на 2 число со знаком, знак числа сохраняется, значение старшего разряда сохраняется прежним (нулем или единицей), младший разряд записывается в качестве флага переноса.

Циклические сдвиги позволяют сдвигать биты кода операнда по кругу, сдвиг вправо - по часовой стрелке, а сдвиг влево - против часовой стрелки. При этом в кольцо сдвига входит флаг переноса. В бит флага переноса записывается значение старшего бита при циклическом сдвиге влево и младшего бита при циклическом сдвиге вправо. Соответственно, значение бита флага переноса будет переписываться в младший разряд при циклическом сдвиге влево и в старший разряд при циклическом сдвиге вправо.

Одноадресные команды модификации данных с регистровой адресацией COM (One's Complement-обратный код), NEG - дополнительный код, INC инкремент (+1), DEC декремент (-1), TST (Test for zero or minus – проверка на ноль и отрицательное число), CLR (Clear Register – очистка регистра), SER (Set all bits in Register- установка всех битов регистра в 1).

Задание 2.2. В программу 2.1 добавьте подпрограмму формирования признака $z=1$, если введенное в регистр r5 произвольное двоичное число соответствует заданной маске. Приведите результаты тестирования программы при пересылке в r5 различных чисел.

Таблица 2.1 – Исходные данные для задания 2.2

1	2	3	4	5	6	7	8
0101xxxx	0011xxxx	1001xxxx	0110xxxx	0111xxxx	1101xxxx	1011xxxx	0001xxxx
9	10	11	12	13	14	15	16
0101xxxx	1110xxxx	0001xxxx	0010xxxx	0100xxxx	1000xxxx	001xxxx0	001xxxx0

Задание 2.3. Составьте подпрограмму, позволяющую исследовать выполнение операций сдвигов. Проведите подробное исследование для различных наборов исходных данных (более чем на 10 различных числах для каждой операции).

Исследование выполнения арифметических операции в микроконтроллерах AVR

Цифровые системы обрабатывают сигналы, представленные в двоичной системе счисления. Один разряд двоичного числа содержит минимальное количество информации, называется «бит» (от Binary Digit - двоичная цифра). Микроконтроллеры AVR обрабатывают данные, содержащие 8 бит или один байт.

Каждая цифра в представлении числа называется разрядом, а соответствующий цифре множитель - весом разряда. С помощью 8-разрядного двоичного кода можно представить $2^8 = 256$ от 0 до $2^n - 1$ (от 0 до 255) чисел. В зависимости от диапазона изменения чисел в конкретной задаче выбирают код для представления этих чисел.

Прямой код используется для записи целых положительных чисел без знака в диапазоне от 0 до $11111111_2 = ff_{16} = 255_{10}$. При суммировании N-разрядных чисел двоичных чисел сумма может иметь N+1 разряд. При появлении дополнительного разряда формируется признак C. Признак C вырабатывается, если в процессе суммирования или вычитания происходит переход через границу между значениями 00000000 и 11111111. Существуют команды, позволяющие использовать этот признак и исключить появление ошибок.

Дополнительный код используется для записи целых чисел со знаком. Старший бит является знаковым для отрицательного числа, он равен 1. Дополнительный код положительного двоичного числа совпадает с прямым кодом, а для отрицательного числа его получают путем прибавления единицы к обратному коду.

Числа в дополнительном коде принимают следующие значения.

Минимальное положительное число 0. Максимальное положительное число 127. Дополнительный код $01111111_2 = 7f_{16} = 127_{10}$.

Минимальное отрицательное число -128. Дополнительный код $10000000_2 = 80_{16} = 128_{10}$.

Максимальное отрицательное число -1. Дополнительный код $11111111_2 = ff_{16} = 255_{10}$.

Для чисел в дополнительном коде признак C указывает на изменение знака при переходе от 0 к -1 или наоборот. Признаком переполнения является переход через границу между положительными и отрицательными числами

от двоичного кода 0111 1111 (соответствует числу +127) к коду 1000 0000 (соответствует числу -128).

Сумма прямого и дополнительного кодов равна 2^n . Прямой код (положительное число) характеризует абсолютное значение отрицательного числа, его модуль М. Для 8-разрядного числа $M + ДК = 2^8$. Например, числу -51 будет соответствовать $ДК = 256 - 51 = 204$.

Задание 2.4. Изучите примеры использования арифметических операций, приведенные в программе LR_3. Для ввода программы создайте новый файл в существующем проекте.

1. Откройте в AVR Studio файл предыдущей работы LR_12 Командой Save As запишите его с именем LR_3.

2. В окне Project установите указатель на Source Files. Из меню по правой кнопке выберите Add Files to Project и укажите LR_3, «Открыть». Имя LR_3 появится в окне Project.

3. В окне Project укажите LR_3, по правой кнопке укажите Set As Entry Files.

4. Удалите старый файл ДК_2.

В отчете необходимо представить результаты выполнения всех программ в виде таблицы, формат которой приведен в работе 1.

Исследуйте выполнение операций суммирования при использовании для представления чисел прямого или дополнительного кода.

Заданы 4 чикла А, В, С, D, записанные в десятичной системе счисления. Исследуйте работу программы, вычисляющей суммы: $A + B$, $B + C$, $C + D$.

Таблица 2.2 – Исходные данные для задания 2.4

Бригада	1	2	3	4	5	6	7	8	9	10
A	51	62	73	84	95	106	17	28	39	99
B	101	102	103	104	105	106	107	108	109	110
C	118	128	138	148	158	168	178	188	198	108
D	231	232	233	234	235	236	237	238	239	240

Двоичные коды, получаемые при суммировании или вычитании, не зависят от выбора кода для представления чисел. Численные результаты операций будут различаться и зависеть от выбора кода для представления чисел (прямой, или дополнительный).

В подпрограмме pp1 предлагается проверить правильность результатов суммирования положительных и знакопеременных чисел. В строках 10 – 13 выполнена инициализация, в регистры r16 – r19 записаны константы. В строках 14 – 16 выполняется суммирование. В программах, содержащих арифметические операции, для упрощения анализа результатов при отладке целесообразно использовать десятичную систему счисления.

1. Включите на выполнение подпрограмму pp1. Символ (;) в начале строки call pp1 должен отсутствовать. Выполните подпрограмму pp1 по шагам, отобразите результаты в виде таблицы, как показано в работе 1.

2. Приведите теоретические значения результатов суммирования и признаков в десятичной и 16-ричной системах счисления для двух вариантов:

- числа положительные, представлены в прямом коде.
- числа со знаком, представлены в дополнительном коде.

3. Сравните экспериментальные результаты с теоретическими. Поясните причины возникновения ошибок.

Задание 2.5. Вычисление среднего значения двух чисел При суммировании М-разрядных чисел сумма может иметь М+1 разряд, дополнительный разряд записывается как признак переполнения C=1. Переполнение при вычитании устанавливает признаки C=1 и N=1.

В подпрограмме pp2 возникновение признака переполнения учитывается командой сдвига с учетом переноса, которая пересылает признак C в старший разряд регистра.

1. Включите на выполнение подпрограмму pp2 и отключите остальные подпрограммы. Выполните подпрограмму pp2 по шагам, отобразите результаты в виде таблицы, как показано в работе 1.

Задание 2.6. Вычисление среднего значения нескольких чисел. При суммировании нескольких слагаемых разрядность суммы возрастает, поэтому для накопления суммы в подпрограмме pp3 предусмотрены два регистра. Размещение операндов в регистрах поясняет рисунок, где X – регистр, содержащий слагаемое (r16 – r19). Переполнения, возникающие при суммировании, учитывается накоплением суммы в r1, при этом слагаемое в r2 равно 0.

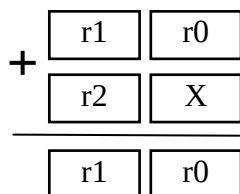


Рисунок 2.2 – Размещение операндов в задании 2.6

Включите на выполнение подпрограмму pp3 и отключите остальные подпрограммы. Выполните подпрограмму pp3 по шагам, отобразите результаты в виде таблицы, как показано в работе 1. Для проверки результатов вычислений, целесообразно отображать данные в окнах AVR Studio в десятичной системе и использовать инженерный калькулятор Windows.

Проверьте работу программы с числами из таблицы задания 3.1 для двух вариантов представления чисел - числа положительные, представлены в прямом коде и числа со знаком, представлены в дополнительном коде.

; LR_3	add r16, r17 ;14	eor r1, r1 ;28
.include "m16def.inc";1	add r17, r18 ;15	eor r2, r2 ;29
ldi r16, 0x04 ;2	add r18, r19 ;16	mov r0, r16 ;30
out sph, r16 ;3	ret ;17	add r0, r17 ;31
ldi r16, 0x5f ;4	pp2: ldi r16, 250 ;18	adc r1, r2 ;32
out spl, r16 ;5	ldi r17, 200 ;19	add r0, r18 ;33
call pp1 ;6	mov r0, r16 ;20	adc r1, r2 ;34
;call pp2 ;7	add r0, r17 ;21	add r0, r19 ;35
;call pp3 ;8	ror r0 ;22	adc r1, r2 ;36
m: jmp m ;9	ret ;23	ror r1 ;37
pp1: ldi r16, 50 ;10	pp3: ldi r16, 250 ;24	ror r0 ;38
ldi r17, 100 ;11	ldi r17, 200 ;25	ror r1 ;39
ldi r18, 150 ;12	ldi r18, 250 ;26	ror r0 ;40
ldi r19, 200 ;13	ldi r19, 200 ;27	ret ;41

Задание 2.7. Умножение положительных чисел. Составьте подпрограмму для исследования команды умножения `mul`, которая перемножает однобайтные положительные сомножители, размещенные в регистрах, указанных в команде. Произведение M - разрядных чисел имеет $2M$ разрядов. Двухбайтное произведение всегда формируется в регистрах `r1:r0`. Запись через двоеточие означает, что отдельные байты объединяются в двухбайтное число, в котором `r1` содержит старший байт результата, а `r0` - младший.

Проверьте работу команды умножения при значениях сомножителей из таблицы задания 2.4.

Задание 2.8. Умножение чисел со знаком. Составьте подпрограмму для исследования команды умножения `muls`, выберите исходные данные, представьте результат работы программы в виде таблицы.

Контрольные вопросы

1. Какие команды обеспечивают использование подпрограмм?
2. Как осуществляется инициализация стека? Как работает стек?
3. Поясните функциональное назначение команд логических операций.
4. Использование логических операций для формирования признаков
5. результата.
6. Использование логических операций с маской.
7. Типы сдвигов, применение команд сдвигов.
8. Регистр состояния процессора. Признаки результата.
9. Команды логических операций с непосредственной адресацией, назначение, функциональные возможности, примеры записи.
10. Команды логических операций с регистровой адресацией, назначение, функциональные возможности, примеры записи.
11. Команды с регистровой и непосредственной адресацией. Поясните на
12. примерах.
13. Косвенная адресация ячейки памяти. Разрядность адреса. Указатели адреса.

14. Косвенная адресация с постинкрементом и преддекрементом. Поясните на
15. примерах.
16. Арифметические операции, способы адресации.
17. Арифметические операции над числами со знаком.
18. Укажи те типы команд умножения, особенности их выполнения, диапазоны чисел.
19. Перечислите команды, использующие признак переноса C.
20. Укажите максимальные значения сомножителей и произведения для команд умножения всех типов.